Project 2. Variational neural network-based solvers and PINNs.

Prepare a single PDF file with your report that includes all relevant figures and numbers. You will use it at the oral defense, and you will submit it to ELMS.

1 Variational Neural Network solvers for the committor problem

The committor problem with the rugged Mueller potential and high-dimensional noise is a popular test problem used e.g. in [1]: Lai and Lu (2017), [2]: Khoo, Lu, Ying (2018), [3]: Li, Lin, Ren (2019) (Section IV, page 6), and [4]: Yuan et al. (2023) (Section 6.2, page 15, Fig. 6). Please take problem parameters from Yuan et al. (2023) (Eqs. (82) and (71)) and training set generation details either from Li et al. (2019) or from Yuan et al. (2023). Set $\beta = 0.1$, which corresponds to temperature $T = 10$.

1. Solve the committor problem using a neural network solver based on the variational formulation, as in Face_NN_deltanet.ipynb.
2. Check your solution using the approach called *committor analysis*. This approach is used when the reference solution is unknown; see, e.g., Li et al. (2019). Do the following.
   - Extract the isocommittor curve $q = 0.5$ of the committor function projected onto the (x_1, x_2)-space.
   - Generate $N = 50$ or 100 points (if your computer is slow, set $N = 50$, if it is alright, set $N = 100$) on it sampled according to the probability measure $e^{-\beta V}$ restricted to this surface. For example, generate many points on this curve and assign weights equal to $e^{-\beta V}$. Sum these weights and divide by their sum to obtain the probability distribution. Then sample N points according to this probability distribution.
   - For each of these N points, launch $M = 200$ stochastic trajectories and terminate them as soon as they reach A or B. For each point, find the probability of reaching B rather than A. This is exactly the committor probability!
   - Plot a histogram of these probability data. If your isocommittor curve $q = 0.5$ is at the right location, the histogram must peak at $p = 0.5$. You can look at an example of such a histogram in [5]: Evans et al (2021) (Fig. 11, page 21).

2 PINNs and spectral methods for the Allen-Cahn PDE

In this project, our test case is the Allen-Cahn PDE, which models phase separation in alloys, including order–disorder transitions. Mathematical properties of this PDE are described, e.g., in [6] (see Section 3.4 and Appendix 7.3).

1. Use PINNs [7]: Raissi, Perdikaris, Karniadakis (2019) to solve the initial-boundary-value problem for the Allen-Cahn PDE (Eq. (12) on page 6 of the PDF). Use the continuous-time setting described in [7] (Section 3.1) and used for the nonlinear Schroedinger equation example in this paper and implemented in NLSE_PINNs.ipynb.
2. Generate a reference solution to the same problem using the spectral method in NLSE_spectral.ipynb.
3. Compare these two solutions by plotting their heatmaps, plotting the solutions at times $t = 0.1$ and $t = 0.9$, and computing the MAE (mean absolute error) and RMSE (root-mean-square) errors.

3 Neural Operators

The goal of this project is to gain experience with neural operators, code them, and adapt them to specific tasks.

We consider the committor problem in 1D on the interval $[-1, 1]$:

$$q''(x) - \beta V'(x) q'(x) = 0, \quad q(-1) = 0, \quad q(1) = 1, \tag{1}$$

where $V(x)$ is a given potential function and $v'(x)$ is its derivative. The inverse temperature parameter β is set to 1. This problem can be readily solved using the Chebyshev spectral method [8] implemented in Committor1D_helpers.py. Our goal is to learn the mapping $G : V'(\cdot) \mapsto q(\cdot)$.

The code DeepONet_1Dcommittor.ipynb implements the neural operator DeepONet [9] and uses

$$u \sim \mathsf{GRF}\left(0, e^{-\frac{|x_1 - x_2|^2}{2l^2}}\right), \tag{2}$$

where the length scale parameter $l = 0.5$. The implementation literally follows the description in [9] except that the final network is run through the sigmoid function to ensure that the solution q belongs to $[0,1]$, as $q(x)$ is the probability for the process starting at $x \in (-1,1)$ to hit first $B := \{x \geq 1\}$ rather than $A := \{x \leq -1\}$:

$$q_{\mathsf{DONet}}(y) = \mathsf{Sigmoid}(b \cdot t), \quad b = \mathsf{BranchNN}(u(\cdot)), \quad t = \mathsf{TrunkNN}(y), \tag{3}$$
$$\mathsf{Sigmoid}(x) = \frac{1}{1+e^{-x}}.$$

I set the dimension of b and t to be $\mathsf{out_size} = 40$.

The input data for the DeepONet were generated as follows. $N_u = 100$ Gaussian random fields defined by (2) were sampled and discretized to $N_x = 100$ equispaced points

$$x_j = -1 + \frac{2j}{(N_x - 1)}, \quad j = 0, \ldots, N_x - 1. \tag{4}$$

We computed the committor for each using the Chebyshev spectral method. Each of these committors was evaluated at $N_{\mathsf{eval}} = 100$ points y sampled from the uniform distribution on $[-1,1]$: $y \sim \mathcal{U}([-1,1])$. Each piece of training data was

$$[u(x_0), u(x_1), \ldots, u(x_{N_x-1}), y, q(y)]. \tag{5}$$

The resulting dataset was of size $N_{\mathsf{train}} = N_u \cdot N_{\mathsf{eval}} = 10,000$.

DeepONet trained successfully and achieved a small MSE on the training data: about $1.36 \cdot 10^{-5}$. However, the generalization error was quite large. The MSE error on the test data sampled from GRF was approximately $3.37 \cdot 10^{-3}$. Furthermore, on the test on the standard double-well potential

$$V(x) = (x^2 - 1)^2, \quad V' = 4x^3 - 4x, \tag{6}$$

the mean squared error was $7.88 \cdot 10^{-3}$. More informative are the corresponding root-mean-squared errors (RMSEs). The RMSE on the GRF test data was $5.81 \cdot 10^{-2}$ and the RMSE for the double-well potential was $8.89 \cdot 10^{-2}$, which is large.

- **Task 1.** Try DeepONet on a simpler parametric committor problem. Consider a three-parameter family of potential functions

$$V(x) = (x^2 - 1)^2 + p_0 e^{-\frac{(x-p_1)^2}{2p_2^2}}, \tag{7}$$

where $p_0 \in [0,5]$, $p_1 \in [-0.5, 0.5]$, and $p_2 \in [0.25, 0.7]$. See DoubleWellPotential.ipynb. Generate a training set where each piece of input data is of the form

$$[p_0, p_1, p_2, y, q(y)] \tag{8}$$

and try to achieve a smaller error on test data. Please feel free to adjust the neural network architecture and the size of the training data. Please feel free to completely rewrite my code.

- **Task 2.** Read the paper introducing the Fourier Neural Operator (FNO) [10]. There is an active GitHub repository with a collection of code:
  https://github.com/neuraloperator/neuraloperator.
  Test FNO on the committor problem with $V(x)$ given by (7) and with $V(x)$ sampled from the GRF. You can implement the FNO on your own or adapt the code from the GitHub repository above.

  Alternatively, you can read the paper introducing IAEnet, the integral Autoencoder Operator network [11], and test IAEnet on the committor problem with $V(x)$ given by (7) and with $V(x)$ sampled from the GRF. The GitHub repository for the IAEnet: https://github.com/IAE-Net/iae_net.

- **Task 3.** Compare the performance of DeepONet and FNO (or IAEnet) on the two committor problems. Plot bar graphs with training RMSE errors and test RMSE errors.

References

[1] R. Lai and J. Lu, "Point cloud discretization of fokker–planck operators for committor functions," *Multiscale Modeling & Simulation*, vol. 16, no. 2, pp. 710–726, 2018.

[2] Y. Khoo, J. Lu, and L. Ying, "Solving for high-dimensional committor functions using artificial neural networks," *Research in the Mathematical Sciences*, vol. 6, p. 1, 2019.

[3] Q. Li, B. Lin, and W. Ren, "Computing committor functions for the study of rare events using deep learning," *The Journal of Chemical Physics*, vol. 151, p. 054112, 08 2019.

[4] J. Yuan, A. Shah, C. Bentz, and M. Cameron, "Optimal control for sampling the transition path process and estimating rates," *Communications in Nonlinear Science and Numerical Simulation*, vol. 129, p. 107701, 2024.

[5] L. Evans, M. K. Cameron, and P. Tiwary, "Computing committors in collective variables via mahalanobis diffusion maps," *Applied and Computational Harmonic Analysis*, vol. 64, pp. 62–101, 2023.

[6] M. Tao, "Hyperbolic periodic orbits in nongradient systems and small-noise-induced metastable transitions," *Physica D: Nonlinear Phenomena*, vol. 363, pp. 1–17, 2018.

[7] M. Raissi, P. Perdikaris, and G. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

[8] A.-K. Kassam and L. N. Trefethen, "Fourth-order time-stepping for stiff pdes," *SIAM Journal on Scientific Computing*, vol. 26, no. 4, pp. 1214–1233, 2005.

[9] L. Lu, P. Jin, Z. Pang, Guofeiand Zhang, and G. E. Karniadakis, "Learning nonlinear operators via deeponet based on the universal approximation theorem of operators," *Nature Machine Intelligence*, vol. 3, pp. 218–229, 2021.

[10] Z.-Y. Li, N. B. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. M. Stuart, and A. Anandkumar, "Fourier neural operator for parametric partial differential equations," *ArXiv*, vol. abs/2010.08895, 2020.

[11] Y. Z. Ong, Z. Shen, and H. Yang, "Integral autoencoder network for discretization-invariant learning," *J. Mach. Learn. Res.*, vol. 23, Jan. 2022.